Problem 1.

1. convert $x = 406.889$ (base-10) into FP given $F = \{\beta = 3, t = 7, L = -7, U = 7\}$ using truncation.

1) Express in base $\beta = 3$: $406 = 120001_3$ and $0.889 = 0.220000002..._3$, so $x = 120001.220000002..._3$
2) Shift to get leading 0, set exponent p: $0.120001220000002... \times 3^6, p = 6$
3) Truncate to t digits: $\text{fl}(x) = 0.1200012 \times 3^6$.

2. what is the value of machine epsilon E for this system?

$E = \beta^{1-t} = 3^{1-7} = 3^{-6} \approx 1.3717 \times 10^{-3}$.

3. what is the positive number with smallest magnitude that can be represented in this system?

0.1000000×3^{-7}.

4. if we were to consider changing the allowable exponent range of F, what is the largest value of U for which overflow would occur when performing the conversion in (1).?

$U = 5$

Problem 2

1. given F(10,5,-10,10) with truncation. Compute the roots of $1.05x^2 + 12.93x + 0.231 = 0$. The true roots are −0.017891 and −12.296. What is the relative error of each of the computed roots?

Carrying out the standard formula by hand in 5-digit truncated arithmetic ($a = 1.05, b = 12.93, c = 0.231$):
$b^2 = 12.93 \times 12.93 = 167.1849 \to 167.18$
$4ac = 4(1.05)(0.231) = 4(0.24255) = 0.97020$
$b^2 - 4ac = 167.18 - 0.97020 = 166.2098 \to 166.20$
$\sqrt{166.20} = 12.89186... \to 12.891$
$x_2 = \frac{-b-\sqrt{b^2-4ac}}{2a} = \frac{-12.93-12.891}{2.10} = -\frac{25.821}{2.10} = -12.29571... \to -12.295$
$x_1 = \frac{-b+\sqrt{b^2-4ac}}{2a} = \frac{-12.93+12.891}{2.10} = -\frac{0.039}{2.10} = -0.01857142... \to -0.018571$ (the subtraction $-12.93 + 12.891$ loses leading digits: cancellation)
By $E_{\text{rel}} = \frac{|x_{\text{exact}} - x_{\text{approx}}|}{|x_{\text{exact}}|}$, we have $E_{\text{rel}_1} = \frac{|-0.017891+0.018571|}{|-0.017891|} \approx 3.8 \times 10^{-2}$ and $E_{\text{rel}_2} = \frac{|-12.296+12.295|}{|-12.296|} \approx 8.1 \times 10^{-5}$.

2. Show a math equivalent formula for x_1 is $x_1 = \frac{2c}{-b-\sqrt{b^2-4ac}}$.

$$
\begin{aligned}
x_1 &= \frac{2c\left(-b+\sqrt{b^2-4ac}\right)}{\left(-b-\sqrt{b^2-4ac}\right)\left(-b+\sqrt{b^2-4ac}\right)} \\
&= \frac{2c\left(-b+\sqrt{b^2-4ac}\right)}{b^2-(b^2-4ac)} \\
&= \frac{2c\left(-b+\sqrt{b^2-4ac}\right)}{4ac} \\
&= \frac{-b+\sqrt{b^2-4ac}}{2a}
\end{aligned}
$$

3. Deduce a algo (avoids cancellation erros) for calculating the roots of a quad. equation.

Algo R:
if $b > 0$ then $x_2 = \frac{-b-\sqrt{b^2-4ac}}{2a}, x_1 = \frac{c}{ax_2}$ else $x_1 = \frac{-b+\sqrt{b^2-4ac}}{2a}, x_2 = \frac{c}{ax_1}$, end

4. redo the calculation in (1) by applying Algo R. Compute the relative error.

By algo R, $x_1 = \frac{c}{ax_2} = -0.017894, x_2 = \frac{-b-\sqrt{b^2-4ac}}{2a} = -12.295$. We have $E_{\text{rel}_1} = \frac{|-0.017891+0.017894|}{|-0.017891|} \approx 1.7 \times 10^{-4}$and $E_{\text{rel}_2} = \frac{|-12.296+12.295|}{|-12.296|} \approx 8.1 \times 10^{-5}$. Compared to (1)'s relative erros, x_1's relative error is much smaller while x_2's unchanged.

Problem 3.

Show that if x is a number in a F, the relative error of the $\text{fl}(x^2-1)$ compared to the true value of x^2-1 is $\leq \left(\frac{|x^2|}{|x^2-1|}+1\right)E$ assuming ignore terms proportional to E^2 and higher powers of E, and no overflow/underflow occurs.

$E_{\text{rel}} = \frac{|x^2-1-\text{fl}(x^2-1)|}{|x^2-1|}$

$= \frac{|x^2-1-\text{fl}(\text{fl}(x^2)-1)|}{|x^2-1|}$

$= \frac{|x^2-1-\text{fl}(x^2(1+\delta_1)-1)|}{|x^2-1|}$

$= \frac{|x^2-1-(x^2(1+\delta_1)-1)(1+\delta_2)|}{|x^2-1|}$

$= \frac{|x^2-1-(x^2+\delta_1 x^2-1)(1+\delta_2)|}{|x^2-1|}$

$= \frac{|x^2-1-(x^2+\delta_1 x^2-1+\delta_2 x^2+\delta_1\delta_2 x^2-\delta_2)|}{|x^2-1|}$

$= \frac{|-x^2\delta_1-(x^2-1)\delta_2-x^2\delta_1\delta_2|}{|x^2-1|}$

$\leq \frac{|-x^2\delta_1|+|-(x^2-1)\delta_2|+|-x^2\delta_1\delta_2|}{|x^2-1|}$ by tri. inequality

$\leq \frac{|-x^2\|\ \delta_1|+|-(x^2-1)\|\delta_2|+|-x^2\ \|\delta_1\|\ \delta_2|}{|x^2-1|}$ by abs. value rules for products

$\leq \frac{|-x^2|E+|-(x^2-1)|E+|-x^2\ |E^2}{|x^2-1|}$ using $|\delta_i| \leq E$

$\leq \frac{|x^2|}{|x^2-1|}(E+E^2)+E$

$\leq \left(\frac{|x^2|}{|x^2-1|}+1\right)E$ ignore terms of power 2 and higher

This gives us a bound on the relative error for the $\text{fl}(x^2-1)$.

Problem 4.

Given u and v, find a cubic polynomial, p(x) such that $p(0)=2, p(1)=6+2u+2v, p'(0)=3v, p''(1)=4u-2v+24$

$p(x)=ax^3+bx^2+cx+d$

Given $2=d\ (1), 6+2u+2v=a+b+c+d\ (2), 3v=c\ (3), 4u-2v+24=6a+2b\ (4)$, we sub $d=2, c=3v$ into 2nd equa.: $6+2u+2v=a+b+3v+2 \rightarrow a+b=4+2u-v\ (5)$. For (4) and (5), we set up:

$$\begin{pmatrix}1 & 1\\ 6 & 2\end{pmatrix}\begin{pmatrix}a\\ b\end{pmatrix}=\begin{pmatrix}2u-v+4\\ 4u-2v+24\end{pmatrix}$$

$$\begin{pmatrix}a\\ b\end{pmatrix}=\begin{pmatrix}1 & 1\\ 6 & 2\end{pmatrix}^{-1}\begin{pmatrix}2u-v+4\\ 4u-2v+24\end{pmatrix}$$

$$\begin{pmatrix}a\\ b\end{pmatrix}=\frac{1}{-4}\begin{pmatrix}2 & -1\\ -6 & 1\end{pmatrix}\begin{pmatrix}2u-v+4\\ 4u-2v+24\end{pmatrix}$$

Solving for $a=4, b=2u-v$. Thus $p(x)=4x^3+(2u-v)x^2+3vx+2$.

Problem 5.

1. Write down the linear system of five equations needed to determine the

derivative values s_1, s_2, s_3, s_4, s_5 for the cubic spline $S(x)$ going through the (x,y) points (−5,2),(0,0), (1,−3),(3,1),(4,2), with a free boundary condition (zero second derivative) at the left end (at x = −5), and a prescribed slope of −1 at the right end (at x = 4). The final linear system should be given in the form $T\vec{s}=\vec{r}$ where T is a matrix, $\vec{s}$ is the vector of unknown slopes s_i, and $\vec{r}$ is the right-hand-

side vector, as described in the course notes, Section 2.5. Show the calculations leading to the entries in your linear system, not just the final system.

i	x_i	y_i	$\Delta x_i = x_{i+1} - x_i$	$y_i' = \frac{y_{i+1}-y_i}{\Delta x_i}$
1	-5	2	5	$-\frac{2}{5}$
2	0	0	1	-3
3	1	-3	2	2
4	3	1	1	1
5	4	2		

By formula (2.5.5) in section 2.5 for internal nodes, $\Delta x_i s_{i-1} + 2(\Delta x_{i-1} + \Delta x_i)s_i + \Delta x_{i-1} s_{i+1} = 3(\Delta x_i y'_{i-1} + \Delta x_{i-1} y'_i)$ for $i = 2, \ldots, n-1$, we have:

$1s_1 + 2(5+1)s_2 + 5s_3 = 3\left(-\frac{2}{5} - 15\right)$

$s_1 + 12s_2 + 5s_3 = -46.2$

$2s_2 + 2(1+2)s_3 + 1s_4 = 3(-6+2)$

$2s_2 + 6s_3 + s_4 = -12$

$1s_3 + 2(2+1)s_4 + 2s_5 = 3(2+2)$

$s_3 + 6s_4 + 2s_5 = 12$

For the left boundary, the free boundary condition is $S_1''(x_1) = 0$. With $h = \Delta x_1 = 5$ and $t = \frac{x - x_1}{h}$, the Hermite form of the spline on $[x_1, x_2]$ is $S_1(t) = (1 - 3t^2 + 2t^3)y_1 + (3t^2 - 2t^3)y_2 + h(t - 2t^2 + t^3)s_1 + h(-t^2 + t^3)s_2$

so $S_1''(t) = \frac{1}{h^2}[(-6 + 12t)y_1 + (6 - 12t)y_2] + \frac{1}{h}[(-4 + 6t)s_1 + (-2 + 6t)s_2]$.

At $t = 0$ (i.e. $x = x_1$) we have $y_1 = 2$, $y_2 = 0$, and $y_1' = \frac{y_2 - y_1}{\Delta x_1} = \frac{0-2}{5} = -\frac{2}{5}$, hence $0 = S_1''(x_1) = \frac{-6y_1 + 6y_2}{h^2} + \frac{-4s_1 - 2s_2}{h} = -\frac{12}{25} - \frac{4s_1 + 2s_2}{5}$

$\rightarrow \frac{4s_1 + 2s_2}{5} = -\frac{12}{25} \rightarrow 4s_1 + 2s_2 = -\frac{12}{5} \rightarrow 2s_1 + s_2 = -1.2$,

which is exactly the notes' free-boundary equation (2.5.8), $s_1 + \frac{s_2}{2} = \frac{3}{2}y_1'$. Multiplying by 5, the first row of the system is $10s_1 + 5s_2 = -6$.

For right boundary, given the right end has a prescribed slope of –1, we have $s_5 = -1$.

Thus, using the $T\vec{s} = \vec{r}$ format gives: $\begin{pmatrix} 10 & 5 & 0 & 0 & 0 \\ 1 & 12 & 5 & 0 & 0 \\ 0 & 2 & 6 & 1 & 0 \\ 0 & 0 & 1 & 6 & 2 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \end{pmatrix} = \begin{pmatrix} -6 \\ -46.2 \\ -12 \\ 12 \\ -1 \end{pmatrix}$

2.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Define the linear system (Hardcoded from Part A)
T = np.array([
    [10.0,  5.0,  0.0,  0.0,  0.0],
    [ 1.0, 12.0,  5.0,  0.0,  0.0],
    [ 0.0,  2.0,  6.0,  1.0,  0.0],
    [ 0.0,  0.0,  1.0,  6.0,  2.0],
    [ 0.0,  0.0,  0.0,  0.0,  1.0]
])
r = np.array([-6.0, -46.2, -12.0, 12.0, -1.0])

# Solve for unknown derivative values s_i
s = np.linalg.solve(T, r)

# Knots data
```

```
x_pts = np.array([-5.0, 0.0, 1.0, 3.0, 4.0])
y_pts = np.array([2.0, 0.0, -3.0, 1.0, 2.0])

# 2. Compute piecewise coefficients
intervals = [
    {"range": (-5, 0), "color": "blue", "marker": "o"},
    {"range": (0, 1),  "color": "green", "marker": "s"},
    {"range": (1, 3),  "color": "red", "marker": "^"},
    {"range": (3, 4),  "color": "purple", "marker": "d"}
]

print("--- CUBIC SPLINE COEFFICIENTS ---")
coeffs = []
for i in range(4):
    h = x_pts[i+1] - x_pts[i]
    dy = y_pts[i+1] - y_pts[i]

    a = y_pts[i]
    b = s[i]
    c = (3.0 * dy / h - 2.0 * b - s[i+1]) / h
    d = (b + s[i+1] - 2.0 * dy / h) / (h**2)
    coeffs.append((a, b, c, d))

    print(f"S{i+1}(x): a={a:8.4f}, b={b:8.4f}, c={c:8.4f}, d={d:8.4f}")

# 3. Visualize smoothly over finely spaced input samples
plt.figure(figsize=(10, 6))

for i in range(4):
    low, high = intervals[i]["range"]
    x_fine = np.linspace(low, high, 100)

    # Evaluate S_i(x) local coordinates relative to the left knot x_i
    x_local = x_fine - x_pts[i]
    a, b, c, d = coeffs[i]
    y_fine = a + b * x_local + c * x_local**2 + d * x_local**3

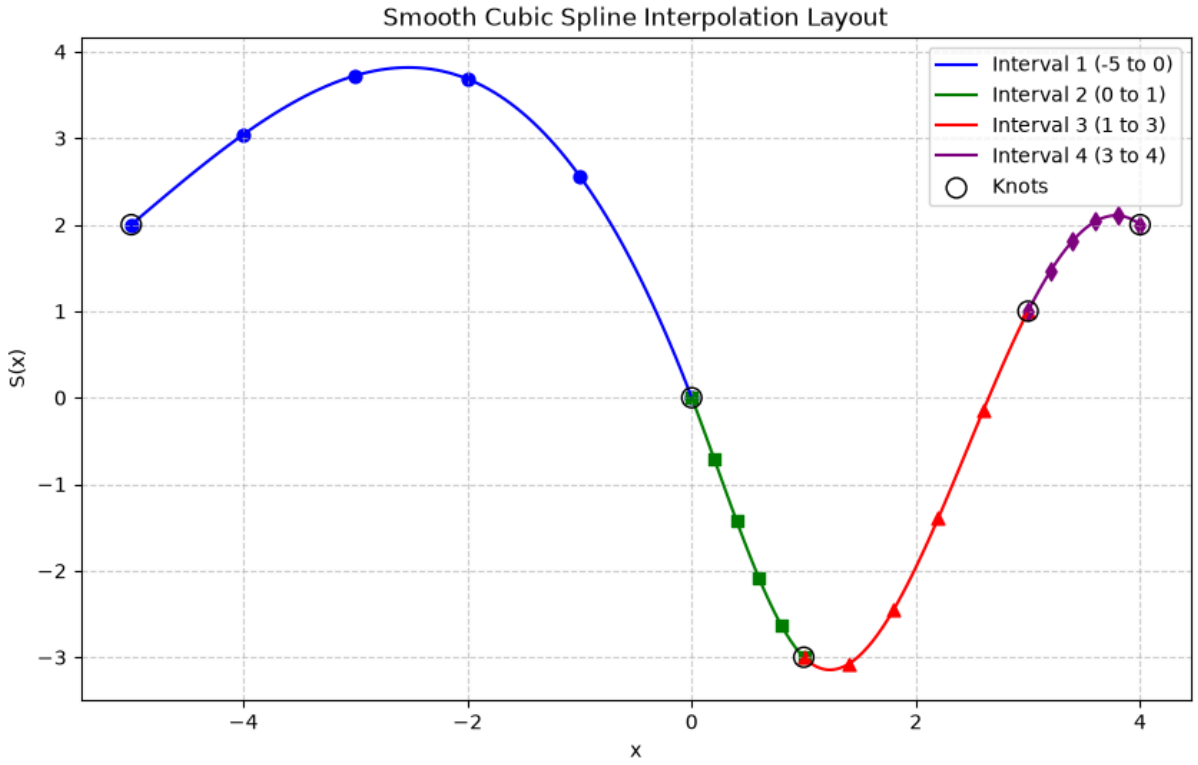
    # Plot interval curve with specific color
    plt.plot(x_fine, y_fine, color=intervals[i]["color"],
             label=f"Interval {i+1} ({low} to {high})")

    # Add markers at sparse evaluation intervals to fit the prompt's layout request
    x_markers = np.linspace(low, high, 6)
    y_markers = a + b * (x_markers - x_pts[i]) + c * (x_markers - x_pts[i])**2 + d *
(x_markers - x_pts[i])**3
    plt.scatter(x_markers, y_markers, color=intervals[i]["color"],
marker=intervals[i]["marker"], s=40)

# Overlay initial target point markers
plt.scatter(x_pts, y_pts, color="black", zorder=5, s=100, facecolors='none',
edgecolors='black', label="Knots")

plt.title("Smooth Cubic Spline Interpolation Layout", fontsize=12)
plt.xlabel("x")
plt.ylabel("S(x)")
plt.grid(True, linestyle="--", alpha=0.6)
```

```
plt.legend()
plt.show()
```



Problem 6

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import make_interp_spline

def collect_curve_datasets():
    curves = []

    curve_idx = 1
    while True:
        fig, ax = plt.subplots(figsize=(6, 4))
        ax.set_title(f"Click points for Curve #{curve_idx} (Press Enter to finish
curve)")
        ax.set_xlim(0, 10)
        ax.set_ylim(0, 5)
        ax.grid(True)

        pts = plt.ginput(n=-1, timeout=0)
        plt.close(fig)

        if len(pts) == 0:
            break

        curves.append(np.array(pts))
        curve_idx += 1
```

```
    return curves


def get_hardcoded_curves():
    c1 = np.array([[2, 4], [1, 3], [1, 1], [2, 0]])            # 'C'
    c2 = np.array([[3, 0], [4, 4], [5, 0], [3.5, 2], [4.5, 2]]) # 'A'
    c3 = np.array([[6, 4], [8, 4], [7, 4], [7, 0]])            # 'T'
    return [c1, c2, c3]


curves = get_hardcoded_curves()
#curves = collect_curve_datasets()

fig1, ax1 = plt.subplots(figsize=(8, 4))
ax1.set_title("Piecewise Linear Curves (Raw Knots)")
ax1.grid(True)
ax1.set_aspect('equal')

fig2, ax2 = plt.subplots(figsize=(8, 4))
ax2.set_title("Smooth Spline Curves (Natural End Conditions)")
ax2.grid(True)
ax2.set_aspect('equal')

for curve in curves:
    x = curve[:, 0]
    y = curve[:, 1]

    ax1.plot(x, y, '-o', linewidth=1.5, markersize=4)

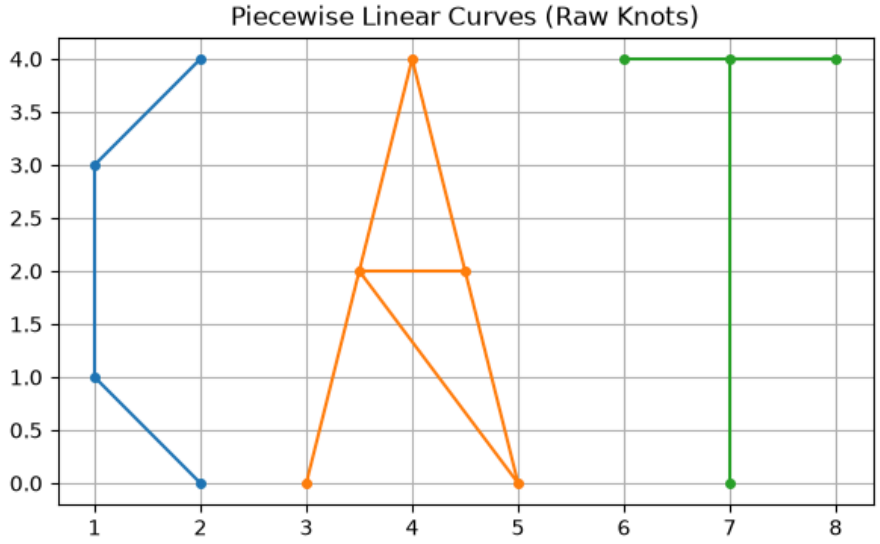
    distances = np.sqrt(np.diff(x)**2 + np.diff(y)**2)
    t = np.zeros(len(x))
    t[1:] = np.cumsum(distances)

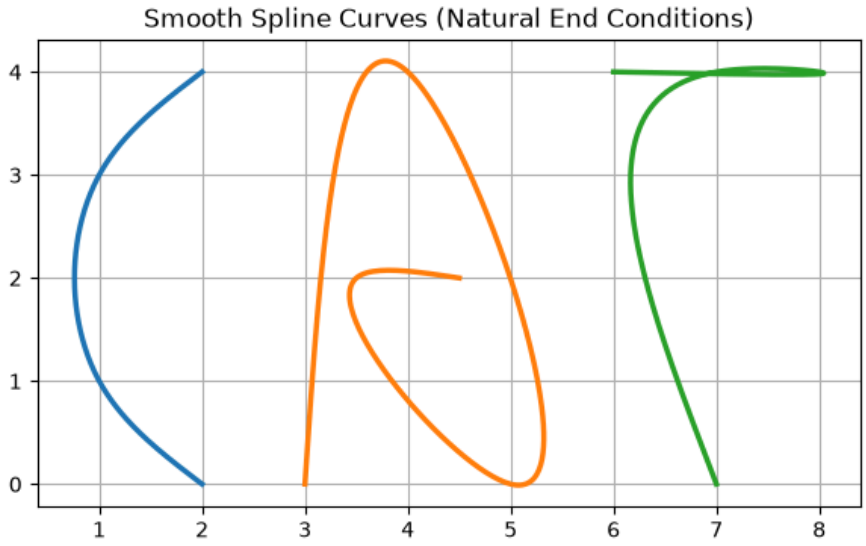
    spl_x = make_interp_spline(t, x, k=3, bc_type='natural')
    spl_y = make_interp_spline(t, y, k=3, bc_type='natural')

    t_fine = np.linspace(t[0], t[-1], 300)
    x_fine = spl_x(t_fine)
    y_fine = spl_y(t_fine)

    ax2.plot(x_fine, y_fine, linewidth=2.5)

plt.show()
```


Piecewise Linear Curves (Raw Knots)
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
1
2
3
4
5
6
7
8


Smooth Spline Curves (Natural End Conditions)
4
3
2
1
0
1
2
3
4
5
6
7
8